Hospital Room Connectivity using Graph and BFS (C++)

Project Title:

Develop a graph representing hospital rooms and corridors, and apply BFS to explore all rooms reachable from the emergency department.

Programming Language:

C++

Data Structure:

Graph (Adjacency List) and Queue

Project Overview:

This project represents hospital rooms as vertices and corridors as edges. The BFS algorithm is used to explore all rooms reachable from the Emergency Department.

Features of the Project:

- Represents hospital rooms and corridors using a Graph.
- Uses an Adjacency List to store room connections.
- Applies Breadth-First Search (BFS) to explore reachable rooms.
- Uses a Queue to process rooms level by level.
- Starts traversal from the Emergency Department.
- Displays all reachable rooms in BFS traversal order.
- Helps understand hospital connectivity and graph traversal in C++.

Technologies Used:

- Programming Language: C++
- Data Structure: Graph (Adjacency List)
- Algorithm: Breadth-First Search (BFS)
- STL Component: Queue
- Development Environment: VS Code / Code::Blocks
- Compiler: GCC (g++)

How to Run the Project:

1. Write the C++ program for the hospital graph using an adjacency list.
2. Implement the BFS algorithm using a queue.
3. Save the source code as hospital_bfs.cpp.
4. Compile the program using a C++ compiler such as GCC (g++).

5. Run the compiled program.
6. The program starts BFS traversal from the Emergency Department.
7. View the output showing all reachable hospital rooms in BFS order.

Problem Statement:

In a hospital, different rooms and departments are connected through corridors. Finding all rooms reachable from the Emergency Department can be difficult when there are many connections. This project uses a Graph data structure and the Breadth-First Search (BFS) algorithm in C++ to explore and display all reachable hospital rooms efficiently.

The system should perform the following operations:

- Create a graph representing hospital rooms and corridors.
- Store room connections using an adjacency list.
- Select the Emergency Department as the starting vertex.
- Apply BFS using a queue to traverse the graph.
- Mark each visited room to avoid revisiting it.
- Display all reachable rooms in BFS traversal order.
- Stop when all reachable rooms have been visited.

Algorithm: Hospital Graph Using BFS:

1. Start
2. Create a graph representing hospital rooms and corridors.
3. Store the connections using an adjacency list.
4. Initialize all rooms as unvisited.
5. Select the Emergency Department as the starting vertex.
6. Insert the starting vertex into the queue and mark it as visited.
7. Remove a room from the queue and display it.
8. Visit all its unvisited adjacent rooms and insert them into the queue.
9. Repeat Steps 7–8 until the queue becomes empty.
10. Stop

Advantages:

- Easy to represent hospital rooms and corridors using a graph.
- Finds all rooms reachable from the Emergency Department.
- BFS explores rooms level by level.
- Uses a queue for systematic traversal.
- Avoids revisiting rooms using a visited array.
- Efficient for traversing connected hospital networks.

- Improves understanding of graph data structures and BFS in C++.

Output:

```
Hospital Room Connectivity

Starting Point: Emergency Department

BFS Traversal:
Emergency Department
Lobby
Pharmacy
Ward A
Ward B
Laboratory

All reachable rooms explored successfully.
```

Result: The system displays all rooms reachable from the Emergency Department in Breadth-First Search (BFS) order.